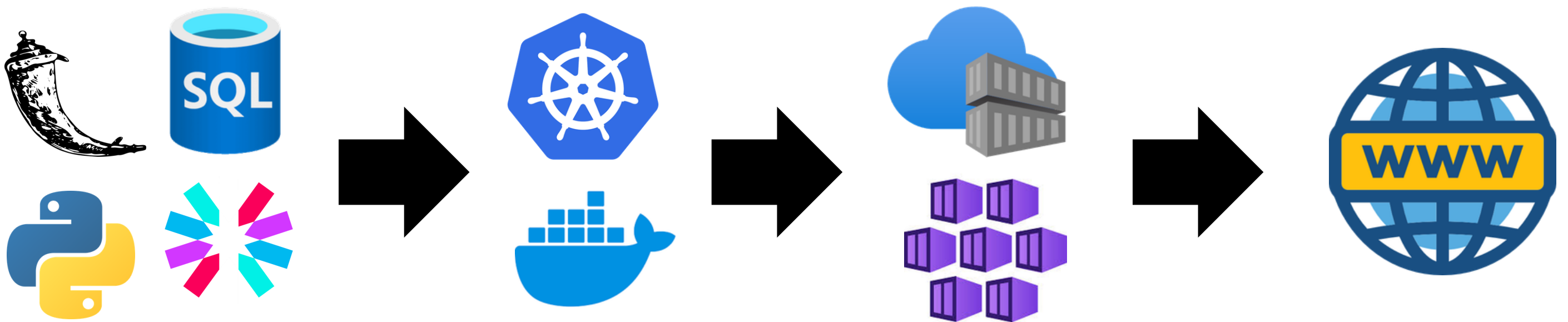


PROJECT DOCUMENTATION

BadgeSys: Secure Badge System using Flask API deployed on Azure Kubernetes Service



Project made by Ayoub Anlouf

Table of contents

I - Creating the backend API using Python Flask

- 1 - Programming the Web login functions
- 2 - Programming the Badge Generation and Validation functions
- 3 - Integrating JWT security Tokens
- 4 - Linking the Azure SQL Database

II - Creating the frontend scripts

- 1 - Functions for communication with the API Endpoints
- 2 - Authentication Context Provider

III - Deploying the app to Azure Container Registry

- 1 - Creating the Dockerfiles
- 2 - Building the images
- 3 - Creating a private Container Registry
- 4 - Pushing the built images to the Registry

IV - Deployed the containerized app to Azure Kubernetes Service

- 1 - Creating an AKS cluster with 2 nodes
- 2 - Connecting Kubectl to the cluster
- 3 - Assigning AcrPull role to the cluster's managed identity
- 4 - Configuring Kubernetes manifest YAML files

V - Visualizing the deployment

- 1 - Visualizing pods status and services
- 2 - Retrieving the Ingress IP Address

Links

Python - <https://www.python.org/downloads/>

AzureCLI - <https://learn.microsoft.com/en-us/cli/azure/install-azure->

Kubectl - <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux>

Docker - <https://docs.docker.com/engine/install/ubuntu/>

NodeJs - <https://nodejs.org/en/download>

I - Creating the backend API using Python Flask

1 - Programming the Web login functions

```
@app.route("/api/register", methods=["POST"])
def register():
    """Register a new user. Body: {username, password}"""
    data = request.get_json(silent=True) or {}
    username = (data.get("username") or "").strip()
    password = data.get("password") or ""

    if not username or not password:
        return jsonify({"error": "username and password are required"}), 400
    if len(username) < 3 or len(username) > 50:
        return jsonify({"error": "username must be 3-50 characters"}), 400
    if len(password) < 6:
        return jsonify({"error": "password must be at least 6 characters"}), 400

    if User.query.filter_by(username=username).first():
        return jsonify({"error": "Username already taken"}), 409

    user = User(username=username)
    user.set_password(password)
    db.session.add(user)
    db.session.commit()

    token = create_token(user.id)
    return jsonify({"message": "User registered successfully", "token": token, "username": username}), 201
```

```
@app.route("/api/login", methods=["POST"])
def login():
    """Authenticate a user. Body: {username, password}"""
    data = request.get_json(silent=True) or {}
    username = (data.get("username") or "").strip()
    password = data.get("password") or ""

    user = User.query.filter_by(username=username).first()
    if not user or not user.check_password(password):
        return jsonify({"error": "Invalid username or password"}), 401

    token = create_token(user.id)
    return jsonify({"message": "Login successful", "token": token, "username": username}), 200
```

2 - Programming the Badge Generation and Validation functions

```
@app.route("/api/generate_badge", methods=["POST"])
@token_required
def generate_badge(current_user):
    """Generate a badge for the authenticated user (one badge per user)."""
    if Badge.query.filter_by(user_id=current_user.id).first():
        return jsonify({"error": "Badge already generated for this account"}), 409

    badge_id = str(uuid.uuid4()).upper()
    image_b64 = generate_badge_image(current_user.username, badge_id)

    badge = Badge(
        badge_id=badge_id,
        user_id=current_user.id,
        image_data=image_b64,
    )
    db.session.add(badge)
    db.session.commit()

    return jsonify({
        "message": "Badge generated successfully",
        "badge_id": badge_id,
        "image": image_b64,
    }), 201
```

```
@app.route("/api/my_badge", methods=["GET"])
@token_required
def my_badge(current_user):
    badge = Badge.query.filter_by(user_id=current_user.id).first()
    if not badge:
        return jsonify({"badge": None}), 200
    return jsonify({
        "badge": {
            "badge_id": badge.badge_id,
            "image": badge.image_data,
            "created_at": badge.created_at.isoformat(),
        }
    }), 200
```

```

@app.route("/api/validate_badge", methods=["POST"])
def validate_badge():
    """
    Validate a badge by ID or uploaded image.
    Accepts multipart/form-data with optional fields:
    - badge_id: text
    - image: file (PNG/JPEG containing QR code)
    """
    badge_id = None

    # Try plain text ID first
    if request.content_type and "multipart" in request.content_type:
        badge_id = (request.form.get("badge_id") or "").strip().upper() or None
        image_file = request.files.get("image")
        if not badge_id and image_file:
            img_bytes = image_file.read()
            badge_id = decode_qr_from_image(img_bytes)
            if not badge_id:
                return jsonify({"valid": False, "error": "Could not decode QR code from image"}), 400
        else:
            data = request.get_json(silent=True) or {}
            badge_id = (data.get("badge_id") or "").strip().upper() or None

    if not badge_id:
        return jsonify({"valid": False, "error": "Provide badge_id or an image with a QR code"}), 400

    badge = Badge.query.filter_by(badge_id=badge_id).first()
    if not badge:
        return jsonify({"valid": False, "message": "Badge not found"}), 200

    user = db.session.get(User, badge.user_id)
    return jsonify({
        "valid": True,
        "badge_id": badge.badge_id,
        "username": user.username if user else "Unknown",
        "issued_at": badge.created_at.isoformat(),
    }), 200

```

3 - Integrating JWT security Tokens

```

def create_token(user_id: int) -> str:
    payload = {
        "sub": str(user_id),
        "exp": datetime.now(timezone.utc) + timedelta(hours=TOKEN_EXP_HOURS),
        "iat": datetime.now(timezone.utc),
    }
    return jwt.encode(payload, app.config["SECRET_KEY"], algorithm=JWT_ALGORITHM)

```



```

def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            return jsonify({"error": "Missing or invalid Authorization header"}), 401
        token = auth_header.split(" ", 1)[1]
        try:
            payload = jwt.decode(
                token, app.config["SECRET_KEY"], algorithms=[JWT_ALGORITHM]
            )
        except jwt.ExpiredSignatureError:
            return jsonify({"error": "Token expired"}), 401
        except jwt.InvalidTokenError:
            return jsonify({"error": "Invalid token"}), 401

        user = db.session.get(User, int(payload["sub"]))
        if user is None:
            return jsonify({"error": "User not found"}), 401
        return f(user, *args, **kwargs)

    return decorated

```

4 - Linking the Azure SQL Database

```

class User(db.Model):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(256), nullable=False)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc), nullable=False)

    # One-to-one: each user has at most one badge
    badge = db.relationship("Badge", back_populates="user", uselist=False, cascade="all, delete-orphan")

    def set_password(self, password: str) -> None:
        self.password_hash = generate_password_hash(password)

    def check_password(self, password: str) -> bool:
        return check_password_hash(self.password_hash, password)

    def __repr__(self) -> str:
        return f"<User {self.username}>"

```

```
class Badge(db.Model):
    __tablename__ = "badges"

    id = db.Column(db.Integer, primary_key=True)
    badge_id = db.Column(db.String(36), unique=True, nullable=False, index=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False, unique=True)
    # Base64-encoded PNG image stored in the database
    image_data = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc), nullable=False)

    user = db.relationship("User", back_populates="badge")

    def __repr__(self) -> str:
        return f"<Badge {self.badge_id}>"
```


II - Creating the frontend scripts

1 - Functions for communication with the API Endpoints

```
export async function register(username: string, password: string): Promise<AuthResponse> {
  const res = await fetch(`${BASE}/register`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });
  return handleResponse<AuthResponse>(res);
}
```

```
export async function login(username: string, password: string): Promise<AuthResponse> {
  const res = await fetch(`${BASE}/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });
  return handleResponse<AuthResponse>(res);
}
```

```
export async function generateBadge(token: string): Promise<BadgeData> {
  const res = await fetch(`${BASE}/generate_badge`, {
    method: "POST",
    headers: authHeaders(token),
  });
  return handleResponse<BadgeData>(res);
}

export async function fetchMyBadge(token: string): Promise<{ badge: BadgeData | null }> {
  const res = await fetch(`${BASE}/my_badge`, {
    headers: authHeaders(token),
  });
  return handleResponse<{ badge: BadgeData | null }>(res);
}
```

```
export async function validateBadgeById(badgeId: string): Promise<ValidationResult> {
  const res = await fetch(`${BASE}/validate_badge`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ badge_id: badgeId }),
  });
  return handleResponse<ValidationResult>(res);
}

export async function validateBadgeByImage(file: File): Promise<ValidationResult> {
  const form = new FormData();
  form.append("image", file);
  const res = await fetch(`${BASE}/validate_badge`, {
    method: "POST",
    body: form,
  });
  return handleResponse<ValidationResult>(res);
}
```

2 - Authentication Context Provider

```
export function AuthProvider({ children }: { children: ReactNode }) {

  const [state, setState] = useState<AuthState>(() => ({
    token: localStorage.getItem("badge_token"),
    username: localStorage.getItem("badge_username"),
  }));

  const setAuth = (token: string, username: string) => {
    localStorage.setItem("badge_token", token);
    localStorage.setItem("badge_username", username);
    setState({ token, username });
  };

  const clearAuth = () => {
    localStorage.removeItem("badge_token");
    localStorage.removeItem("badge_username");
    setState({ token: null, username: null });
  };

  return (
    <AuthContext.Provider value={{ ...state, setAuth, clearAuth }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

```
const AuthContext = createContext<AuthContextValue>({
  token: null,
  username: null,
  setAuth: () => {},
  clearAuth: () => {},
});
```

III - Deploying the app to Azure Container Registry

1 - Creating the Dockerfiles

```
GNU nano 7.2 Dockerfile *
FROM node:20 AS build
WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

# Set API URL for production build
ENV VITE_API_URL=/api
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line
```

```
GNU nano 7.2 backend/Dockerfile *
FROM python:3.12-slim

WORKDIR /app

# Install prerequisites
RUN apt-get update && apt-get install -y \
    curl gnupg wget unzip apt-transport-https \
    libzbar0 unixodbc unixodbc-dev gcc libgssapi-krb5-2 \
    && rm -rf /var/lib/apt/lists/*

# Add Microsoft repo key and list (Debian 12 / bookworm)
RUN curl https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor -o /usr/share/keyrings/microsoft-prod.gpg \
    && echo "deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft-prod.gpg] https://packages.microsoft.com/debian/12>
    > /etc/apt/sources.list.d/mssql-release.list

# Install ODBC Driver 18 and tools
RUN apt-get update \
    && ACCEPT_EULA=Y apt-get install -y msodbcsql18 mssql-tools18 \
    && echo 'export PATH="/opt/mssql-tools18/bin"' >> /etc/bash.bashrc \
    && rm -rf /var/lib/apt/lists/*

ENV PATH="/opt/mssql-tools18/bin"

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location  M-U Undo    M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo    M-6 Copy
```


2 - Building the images

```
ubuntu@ubuntu:~$ cd BadgeSys
ubuntu@ubuntu:~/BadgeSys$ docker image build -t badgesys-backend:v1 ./backend
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

                Install the buildx component to build images with BuildKit:
                https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  19.46kB
Step 1/7 : FROM python:3.12-slim
3.12-slim: Pulling from library/python
3fc9d9ab5045: Pulling fs layer
3531af2bc2a9: Pulling fs layer
cb8c0a9140ac: Pulling fs layer
4d873bcef452: Pulling fs layer
3fc9d9ab5045: Download complete
75cf1a72ec4f: Download complete
cb8c0a9140ac: Download complete
67a6ed3b38af: Download complete
4d873bcef452: Download complete
3531af2bc2a9: Download complete
3531af2bc2a9: Pull complete
cb8c0a9140ac: Pull complete
3fc9d9ab5045: Pull complete
4d873bcef452: Pull complete
```

```
ubuntu@ubuntu:~/BadgeSys$ docker image build -t badgesys-frontend:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

                Install the buildx component to build images with BuildKit:
                https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  337.9kB
Step 1/10 : FROM node:20 AS build
20: Pulling from library/node
0b22ec2d9b1a: Pulling fs layer
db53381ee51f: Pulling fs layer
e5203b2bfeff: Pulling fs layer
8fcda2b4d799: Pulling fs layer
2933ecab0f11: Pulling fs layer
c83419f44309: Pulling fs layer
76e6b8204a89: Pulling fs layer
2f2ab6e4eb46: Pulling fs layer
0b22ec2d9b1a: Download complete
3e22c7cc721e: Download complete
c83419f44309: Download complete
```

```
ubuntu@ubuntu:~/BadgeSys$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
badgesys-backend:v1	556090156ae8	263MB	63.7MB	
badgesys-frontend:v1	384cd7e99409	92.8MB	26.1MB	
nginx:alpine	5616878291a2	93.5MB	26.9MB	
node:20	8f693eaa7e0a	1.59GB	415MB	
python:3.12-slim	520153e2deb3	179MB	45.4MB	

```
ubuntu@ubuntu:~/BadgeSys$
```

3 - Creating a private Container Registry

```
PS C:\Users\Msi\Desktop\BadgeSys> az acr create --resource-group BadgeSysRG --name badgesysregistry --sku Basic
{
  "adminUserEnabled": false,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-04-22T16:36:19.597533+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "location": "West Europe",
  "name": "badgesysregistry",
  "resourceGroup": "BadgeSysRG",
  "sku": "Basic",
  "status": "Succeeded"
}
```

```
ubuntu@ubuntu:~/BadgeSys$ az acr login --name badgesysregistry
Login Succeeded
ubuntu@ubuntu:~/BadgeSys$
```

4 - Pushing the built images to the Registry

```
ubuntu@ubuntu:~/BadgeSys$ docker tag badgesys-backend:v1 badgesysregistry.azurecr.io/badgesys-backend:v1
ubuntu@ubuntu:~/BadgeSys$ docker tag badgesys-frontend:v1 badgesysregistry.azurecr.io/badgesys-frontend:v1
ubuntu@ubuntu:~/BadgeSys$
```

```
ubuntu@ubuntu:~/BadgeSys$ docker push badgesysregistry.azurecr.io/badgesys-backend:v1
The push refers to repository [badgesysregistry.azurecr.io/badgesys-backend]
3531af2bc2a9: Pushed
cb8c0a9140ac: Pushed
43924706966e: Pushed
3fc9d9ab5045: Pushed
e4c88a9e4486: Pushed
361310a045f2: Pushed
5df884fe8587: Pushed
4d873bcef452: Pushed
v1: digest: sha256:556090156ae85457223e7d7a7c2f9e21906a7c803e436d72e853e0ad69e107e2 size: 2056
```

```
ubuntu@ubuntu:~/BadgeSys$ docker push badgesysregistry.azurecr.io/badgesys-frontend:v1
The push refers to repository [badgesysregistry.azurecr.io/badgesys-frontend]
6a0ac1617861: Pushed
82736a35d0e7: Pushed
20f822850fbe: Pushed
aee4e54b3865: Pushed
781ff50d2644: Pushed
453da7dbc73e: Pushed
583599bb7d38: Pushed
4a8b0b2a5b19: Pushed
612c0c1df4c5: Pushed
v1: digest: sha256:384cd7e994091433c2e8d24dc917a26edc9eb6946ff94991eae2d27a5f150313 size: 2263
ubuntu@ubuntu:~/BadgeSys$
```


IV - Deployed the containerized app to Azure Kubernetes Service

1 - Creating an AKS cluster with 2 nodes

```
ubuntu@ubuntu:~/BadgeSys$ az aks create \
  --resource-group BadgeSysRG \
  --name BadgeSysCluster \
  --node-count 2 \
  --node-vm-size Standard_Fx2ms_v2 \
  --enable-addons monitoring \
  --generate-ssh-keys
{
  "aadProfile": null,
  "addonProfiles": {
    "omsagent": {
      "config": {
        "logAnalyticsWorkspaceResourceID": "/subscriptions/afd82d75-3dfb-4266-badc-e0c5afd8b8d1/resourceGroups/DefaultResourceGroup-PAR/providers/Microsoft.OperationalInsights/workspaces/DefaultWorkspace-afd82d75-3dfb-4266-badc-e0c5afd8b8d1-PAR",
        "useAADAuth": "true"
      },
      "enabled": true,
      "identity": null
    }
  },
  "agentPoolProfiles": [
    {
```

2 - Connecting Kubectl to the cluster

```
ubuntu@ubuntu:~/BadgeSys$ az aks get-credentials --resource-group BadgeSysRG --name BadgeSysCluster
Merged "BadgeSysCluster" as current context in /home/ubuntu/.kube/config
ubuntu@ubuntu:~/BadgeSys$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
aks-nodepool1-33068459-vmss000000  Ready    <none>   3m4s   v1.34.4
aks-nodepool1-33068459-vmss000001  Ready    <none>   3m6s   v1.34.4
ubuntu@ubuntu:~/BadgeSys$
```

3 - Assigning AcrPull role to the cluster's managed identity

```
ubuntu@ubuntu:~/BadgeSys$ az aks update \
  --resource-group BadgeSysRG \
  --name BadgeSysCluster \
  --attach-acr badgesysregistry
AAD role propagation done[#####] 100.000
0%
```



```
ubuntu@ubuntu:~/BadgeSys$ az aks show \
  --resource-group BadgeSysRG \
  --name BadgeSysCluster \
  --query identityProfile.kubeletidentity.objectId -o tsv
a9ae17b9-2f8a-45c6-b516-bad4dc075cfd
```

```
ubuntu@ubuntu:~/BadgeSys$ az acr show --name badgesysregistry --query id -o tsv

/subscriptions/afd82d75-3dfb-4266-badc-e0c5afd8b8d1/resourceGroups/BadgeSysRG/providers/Microsoft.ContainerRegistry/registries/badgesysregistry
```

```
ubuntu@ubuntu:~/BadgeSys$ az role assignment create \
  --assignee a9ae17b9-2f8a-45c6-b516-bad4dc075cfd \
  --role AcrPull \
  --scope /subscriptions/afd82d75-3dfb-4266-badc-e0c5afd8b8d1/resourceGroups/BadgeSysRG/providers/Microsoft.ContainerRegistry/registries/badgesysregistry

{
  "condition": null,
  "conditionVersion": null,
  "createdBy": "0680482f-2bd4-4ac0-8bb2-095505142018",
  "createdOn": "2026-04-22T18:59:32.780409+00:00",
  "delegatedManagedIdentityResourceId": null,
  "description": null,
  "id": "/subscriptions/afd82d75-3dfb-4266-badc-e0c5afd8b8d1/resourceGroups/BadgeSysRG/providers/Microsoft.ContainerRegistry/registries/badgesysregistry/providers/Microsoft.Authorization/roleAssignments/75f9d15c-e33a-4e6d-a845-f9f3c25a9e48",
  "name": "75f9d15c-e33a-4e6d-a845-f9f3c25a9e48",
  "principalId": "a9ae17b9-2f8a-45c6-b516-bad4dc075cfd",
  "principalName": "6258d478-a5b3-4dc5-9114-f79f15268013",
  "principalType": "ServicePrincipal",
  "resourceGroup": "BadgeSysRG",
  "roleDefinitionId": "/subscriptions/afd82d75-3dfb-4266-badc-e0c5afd8b8d1/providers/Microsoft.Authorization/roleDefinitions/7f951dda-4ed3-4680-a7ca-43fe172d538d",
  "roleDefinitionName": "AcrPull",

```

4 - Configuring Kubernetes manifest YAML files

```
GNU nano 7.2                                backend-deployment.yaml *
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: badgesysregistry.azurecr.io/badgesys-backend:v1
          ports:
            - containerPort: 5000
          env:
```

```
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
GNU nano 7.2                                frontend-deployment.yaml *
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: badgesysregistry.azurecr.io/badgesys-frontend:v1
          ports:
            - containerPort: 80
          ---
```

```
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
GNU nano 7.2                                ingress.yaml *
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: badgesys-ingress
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  rules:
  - http:
    paths:
    - path: /api
      pathType: Prefix
      backend:
        service:
          name: backend-service
          port:
            number: 5000
    - path: /
      pathType: Prefix
      backend:
```

^G Help **^O** Write Out **^W** Where Is **^K** Cut **^T** Execute **^C** Location
^X Exit **^R** Read File **^_** Replace **^U** Paste **^J** Justify **^/** Go To Line

```
ubuntu@ubuntu:~/BadgeSys$ nano backend-deployment.yaml
ubuntu@ubuntu:~/BadgeSys$ nano frontend-deployment.yaml
ubuntu@ubuntu:~/BadgeSys$ nano ingress.yaml
```

```
ubuntu@ubuntu:~/BadgeSys$ kubectl apply -f backend-deployment.yaml
deployment.apps/backend-deployment created
service/backend-service created
deployment.apps/frontend-deployment created
service/frontend-service created
Warning: annotation "kubernetes.io/ingress.class" is deprecated, please use 'spec.ingressClassName' instead
ingress.networking.k8s.io/badgesys-ingress created
ubuntu@ubuntu:~/BadgeSys$
```



```
ubuntu@ubuntu:~/BadgeSys$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml
namespace/ingress-nginx created
serviceaccount/ingress-nginx created
serviceaccount/ingress-nginx-admission created
role.rbac.authorization.k8s.io/ingress-nginx created
role.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrole.rbac.authorization.k8s.io/ingress-nginx created
clusterrole.rbac.authorization.k8s.io/ingress-nginx-admission created
rolebinding.rbac.authorization.k8s.io/ingress-nginx created
rolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
configmap/ingress-nginx-controller created
service/ingress-nginx-controller created
service/ingress-nginx-controller-admission created
deployment.apps/ingress-nginx-controller created
job.batch/ingress-nginx-admission-create created
job.batch/ingress-nginx-admission-patch created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/ingress-nginx-admission created
ubuntu@ubuntu:~/BadgeSys$
```

```
GNU nano 7.2 backend-secrets.yaml *
apiVersion: v1
kind: Secret
metadata:
  name: backend-secrets
type: Opaque
data:
  SECRET_KEY: <base64-encoded-secret>
  DATABASE_URL: <base64-encoded-db-url>

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
ubuntu@ubuntu:~/BadgeSys$ nano backend-secrets.yaml
ubuntu@ubuntu:~/BadgeSys$ kubectl apply -f backend-secrets.yaml
secret/backend-secrets created
ubuntu@ubuntu:~/BadgeSys$
```

V - Visualizing the deployment

1 - Visualizing pods status and services

```
ubuntu@ubuntu:~/BadgeSys$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-deployment-668fc45dc6-9lmtj 1/1     Running   0           12s
frontend-deployment-5fdbdfd55d-lfxln 1/1     Running   0           3h13m
ubuntu@ubuntu:~/BadgeSys$
```

```
ubuntu@ubuntu:~/BadgeSys$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
backend-service     ClusterIP   10.0.150.71  <none>        5000/TCP   3h20m
frontend-service    ClusterIP   10.0.192.17  <none>        80/TCP     3h20m
kubernetes           ClusterIP   10.0.0.1     <none>        443/TCP    3h33m
ubuntu@ubuntu:~/BadgeSys$
```

2 - Retrieving the Ingress IP Address

```
ubuntu@ubuntu:~/BadgeSys$ kubectl describe ingress badgesys-ingress
Name:                 badgesys-ingress
Labels:               <none>
Namespace:            default
Address:              4.251.145.243
Ingress Class:        <none>
Default backend:      <default>
Rules:
  Host      Path  Backends
  ----      -
  *
            /api  backend-service:5000 (10.244.1.28:5000)
            /      frontend-service:80 (10.244.0.182:80)
Annotations:  kubernetes.io/ingress.class: nginx
Events:       <none>
```